Philosophy of GSP_API Packages

1. Preamble

The repo is split across **seven Python packages** under src/. The split is not arbitrary — each package occupies a specific layer of the architecture, and dependencies flow strictly downward. This document names the layers, walks each package, and shows how they cooperate.

Audience. A contributor opening this repo for the first time, or one returning after months away.

The two-sentence rule that governs the split:

> *Each package should be understandable alone, and the system should be understandable as a whole.*

How to verify any claim. Every section ends with a grep recipe or a file pointer. The code is the source of truth; this document just names what it does. Suggested commands at any point:

```
# Who depends on whom?
grep -rn "^from gsp\|^import gsp" src/<pkg>/ | grep -oE "gsp[a-z_]*" | sort -u

# What contracts does a backend implement?
grep -rn "class .*\(RendererBase\)\|class .*\(AnimatorBase\)\|class .*\
(ViewportEventsBase\)" src/

# Which backends are registered?
grep -rn "RendererRegistry.register_renderer" src/
```

---

2. The Layered Architecture

Three tiers, each smaller in scope than the one above it.

2.1 Contract layer

`gsp` — the abstract protocol. Defines what a Canvas, Viewport, Visual, Buffer, and Renderer *are*, with no commitment to how rendering happens. Imports from no sibling package.

2.2 Backend layer

Three packages, each implementing the abstract bases of `gsp`. They self-register at import time so callers can request a renderer by name without taking a build-time dependency on it.

Package	Backend	Target use case
`gsp_matplotlib`	matplotlib	Static images, CI, publication-quality PNG/SVG/PDF
`gsp_datoviz`	datoviz (GPU)	Interactive, real-time, GPU-accelerated
`gsp_network`	HTTP client + Flask server	Remote rendering; server delegates to one of the other two backends

2.3 Convenience / facade layer

Three packages that build on `gsp` *without* taking a dependency on any backend. They are pure CPU-side helpers — flatteners, serializers, higher-level APIs.

Package	Role
gsp_extra	Helpers above the protocol: `Object3D` scene-graph hierarchy, camera controls, `Bufferx` numpy bridge
gsp_pydantic	Round-trip the entire scene through pydantic models (for sessions, network, snapshot tests)
vispy2	High-level matplotlib-like facade: `scatter`, `plot`, `imshow`, and the `AxesManaged` / `AxesDisplay` / `AxesPanZoom` stack

2.4 The golden rule: dependencies point downward

From → To	gsp	gsp_matplotlib	gsp_datoviz	gsp_network	gsp_extra	gsp_py
gsp	—	×	×	×	×	×
gsp_matplotlib	✓	—	×	×	×	×
gsp_datoviz	✓	×	—	×	×	×
gsp_network	✓	×	×	—	×	✓
gsp_extra	✓	×	×	×	—	×
gsp_pydantic	✓	×	×	×	×	—
vispy2	✓	×	×	×	✓	×

Read this table as a contract:

- `gsp` is the only package every other package imports.
- Backends never depend on each other.
- Convenience packages never depend on backends — they speak to the abstract types only.
- The lone "horizontal" edge is `gsp_network → gsp_pydantic`: the network backend uses serialization to put scenes on the wire.

The table describes the **renderer-side** dependency graph — the part of each package a user imports to render. The `gsp_network` package additionally ships an executable Flask server under tools/ that does import both backends to dispatch incoming requests. That tool is a deployment artefact, not a library dependency — see §3.4.

This is what makes the env-var swap `GSP_RENDERER=matplotlib|datoviz|network` actually work — see philosophy_examples.md §2.1.

---

3. Per-Package Reference

Each entry follows the same template: **What it is** → **How to use it** → **How it fits in the system**, with optional configuration / extension / stability notes.

3.1 `gsp` — the contract

What it is. The backend-agnostic protocol for describing 2D/3D scenes. Concrete data containers (`Canvas` , `Viewport` , `Camera` , `Buffer` , `Points` , `Mesh` , ...) plus abstract base classes (`RendererBase` , `AnimatorBase` , `ViewportEventsBase` , `SerializerBase`) that backends implement.

Boundaries. Defines **no rendering**. Imports from numpy/stdlib only — verifiable with:

```
grep -rn "^from gsp_\|^import gsp_" src/gsp/   # no matches
```

Subpackage tour.

Subpackage	Role	Key exports
`gsp.core`	Scene-graph containers	`Canvas`, `Viewport`, `Camera`, `Texture`, `Event`
`gsp.visuals`	Concrete visuals (data containers, not draw commands)	`Points`, `Pixels`, `Markers`, `Paths`, `Segments`, `Texts`, `Image`, `Mesh`
`gsp.types`	Abstract bases + value types	`RendererBase`, `VisualBase`, `AnimatorBase`, `ViewportEventsBase`, `SerializerBase`, `Buffer`, `BufferType`, `Color`, `MarkerShape`, `CapStyle`, `JoinStyle`, `TextAlign`, `ImageInterpolation`, `Group`, `TransBuf`
`gsp.transforms`	`TransformChain` and registry	`TransformChain`, `TransformLinkBase`, `TransformRegistry`
`gsp.geometry`	Geometry containers	`Geometry`, `MeshGeometry`
`gsp.materials`	Material descriptors	`Material`, `MeshBasicMaterial`, `MeshMaterial`
`gsp.utils`	Free functions and the registry	`RendererRegistry`, `CmapUtils`, `GroupUtils`, `UnitUtils`, `ViewportUnitUtils`, `MathUtils`, `UuidUtils`

Key entry points. Most user code touches:

```python
from gsp.core import Canvas, Viewport, Camera
from gsp.visuals import Points, Pixels, Markers, Paths, Mesh
from gsp.types.buffer import Buffer
from gsp.types.buffer_type import BufferType
from gsp.utils.renderer_registery import RendererRegistry
from gsp.constants import Constants
```

A note on imports: [src/gsp/types/**init**.py](src/gsp/types/__init__.py) deliberately does not re-export `RendererBase` or `SerializerBase` (to avoid a circular import). Import them from the submodule instead:

```python
from gsp.types.renderer_base import RendererBase
from gsp.types.serializer_base import SerializerBase
```

Extension point — `RendererRegistry` . This is the seam through which backends plug in. The registry stores a `(name, RendererBase, ViewportEventsBase, AnimatorBase)` tuple per backend and exposes three factories:

```
RendererRegistry.create_renderer(name, canvas)          # → RendererBase
RendererRegistry.create_viewport_events(renderer, vp)   # → ViewportEventsBase
RendererRegistry.create_animator(renderer)              # → AnimatorBase
```

Definition: src/gsp/utils/renderer_registery.py:24-100. Every backend's `renderer_registration.py` calls `register_renderer(...)` to add itself.

3.2 `gsp_matplotlib` — static / CI backend

What it is. The default backend. Renders to PNG/SVG/PDF via matplotlib. Needs no GPU; ideal for CI, headless servers, and publication output.

Implements.

Contract	Class	File
`RendererBase`	`MatplotlibRenderer`	renderer/matplotlib_renderer.py:38
`AnimatorBase`	`AnimatorMatplotlib`	animator/animator_matplotlib.py:37
`ViewportEventsBase`	`ViewportEventsMatplotlib`	viewport_events/viewport_events_matplotlib.py:22

Per-visual rendering is split across one file per visual type (`matplotlib_renderer_points.py` , `matplotlib_renderer_mesh.py` , ...) — same pattern in every backend.

How to use it.

```
import gsp_matplotlib                                          # triggers self-registration
from gsp.utils.renderer_registery import RendererRegistry
renderer = RendererRegistry.create_renderer("matplotlib", canvas)
png_bytes = renderer.render([viewport], [visual], [model_matrix], [camera])
```

The matplotlib backend additionally accepts `image_format="png"|"svg"|"pdf"` for vector output — see examples/svg_pdf_example.py.

Registration. renderer_registration.py:10-17:

```
RendererRegistry.register_renderer(
    renderer_name="matplotlib",
    renderer_base_type=MatplotlibRenderer,
    viewport_event_base_type=ViewportEventsMatplotlib,
    animator_base_type=AnimatorMatplotlib,
)
```

3.3 `gsp_datoviz` — GPU / interactive backend

What it is. Real-time GPU rendering via the datoviz library. Target: interactive 3D, large point clouds, smooth animation.

Implements.

Contract	Class	File
`RendererBase`	`DatovizRenderer`	renderer/datoviz_renderer.py:41
`AnimatorBase`	`AnimatorDatoviz`	animator/animator_datoviz.py:19
`ViewportEventsBase`	`ViewportEventsDatoviz`	viewport_events/viewport_events_datoviz.py:11

Same per-visual file split as the matplotlib backend.

How to use it. Identical pattern to §3.2, just substitute the name:

```
import gsp_datoviz
renderer = RendererRegistry.create_renderer("datoviz", canvas)
```

Registration. renderer_registration.py:10-17.

3.4 `gsp_network` — remote rendering

What it is. A two-piece package for rendering across the network: a thin HTTP client (`NetworkRenderer`) on the user side, and a Flask server (`tools/network_server.py`) on the rendering host.

Implements.

Contract	Class	File
`RendererBase`	`NetworkRenderer`	renderer/network_renderer.py:38
`AnimatorBase`	`AnimatorNetwork`	animator/animator_network.py:26
`ViewportEventsBase`	`ViewportEventsNetwork`	viewport_events/viewport_events_network.py:15

How to use it (client side).

```
from gsp_network.renderer.network_renderer import NetworkRenderer
renderer = NetworkRenderer(canvas, server_base_url="http://localhost:5000",
                           remote_renderer_name="datoviz")  # or "matplotlib"
png_bytes = renderer.render([viewport], [visual], [model_matrix], [camera])
```

The client serializes the scene with `gsp_pydantic` , POSTs it, and returns the rendered bytes.

How to use it (server side).

```
python -m gsp_network.tools.network_server
```

Server picks a backend **per request**, reading `payload["renderer_name"]` at network_server.py:75-79:

```
if renderer_name == "matplotlib":
    renderer = MatplotlibRenderer(parsed_canvas)
else:
    renderer = DatovizRenderer(parsed_canvas, offscreen=True)
```

This is the only place in the codebase where a backend is mentioned by name outside its own package or its registration file — and even here it's an isolated dispatch, not a structural dependency.

Dependency note. `gsp_network` is the only backend that depends on `gsp_pydantic` (it has to — that's the wire format).

3.5 `gsp_pydantic` — full-scene serialization

What it is. Bidirectional translation between a live GSP scene (canvas + viewports + visuals + buffers + transforms + cameras) and a tree of pydantic models. Round-trips through JSON.

Boundaries. Serialization only. No rendering, no state, no UI.

Key entry points (serializer/**init**.py:3-4):

```
from gsp_pydantic.serializer.pydantic_serializer import PydanticSerializer
from gsp_pydantic.serializer.pydantic_parser import PydanticParser
```

Usage shape.

```
serializer = PydanticSerializer(canvas)
serialized: PydanticDict = serializer.serialize(viewports=[...], visuals=[...],
                                                model_matrices=[...], cameras=[...])

parser = PydanticParser()
_, viewports, visuals, model_matrices, cameras = parser.parse(serialized)
```

Two consumers.

1. The `gsp_network` server, which deserializes incoming payloads.
2. The session record/replay examples — see philosophy_examples.md §4.8 and examples/session_record_example.py.

3.6 `gsp_extra` — convenience helpers

What it is. A grab-bag of helpers built on `gsp` that almost every non-trivial example needs but that don't belong in the core protocol.

Boundaries. No rendering. No backend awareness. Pure CPU-side data manipulation above the abstract types.

Key entry points.

Helper	What it does	Where
`Object3D`	Scene-graph node with parent/child links and Euler-angle local transforms. `Object3D.pre_render(viewport, scene, camera)` flattens a hierarchy into the four parallel lists `RendererBase.render(...)` consumes.	object3d.py:20, `pre_render` at object3d.py:229
`Bufferx`	The standard numpy → GPU bridge. `Bufferx.from_numpy(arr, BufferType.vec3)` and	bufferx.py

	`Bufferx.mat4_identity()` show up in every example.	
`AwsdControls`, `TrackballControls`	Camera control schemes that mutate an `Object3D` from `ViewportEvents`.	camera_controls/
`RenderItem`	A bundle of `(viewport, visual, model_matrix, camera)` — useful when you want to manipulate render entries as a list rather than four parallel ones.	misc/render_item.py
`TextureUtils`, `MeshUtils`	Image loaders, mesh helpers.	misc/
Transform links	`TransformLoad` and friends — building blocks for `TransformChain`.	transform_links/

The `Object3D.pre_render` bridge. The renderer's contract is *four parallel lists*; an `Object3D` tree is hierarchical. `pre_render` is the adapter — see philosophy_examples.md §4.5 for the canonical usage. It lets you build scenes with `add()` / `remove()` / `attach_visual()` and still call the unchanged renderer API.

3.7 `vispy2` — high-level facade

What it is. A matplotlib-like convenience API for users who want `scatter()` and `imshow()` rather than the raw four-list `renderer.render(...)` call. Hosts the `AxesManaged` / `AxesDisplay` / `AxesPanZoom` stack used by the interactive examples.

Key entry points.

Module	Exports
`vispy2.axes`	`AxesManaged`, `AxesDisplay`, `AxesPanZoom`, `AxisTickLocator`, `AxisTickFormatter`
`vispy2.scatter`	`scatter(...)`
`vispy2.plot`	`plot(...)`
`vispy2.imshow`	`imshow(...)`

The Axes ladder (rendering / state / interaction, separable):

Class	Purpose	When to reach for it
`AxesManaged`	Wraps a viewport, auto-handles pan/zoom/labels/title.	Tutorial code, quick demos.
`AxesDisplay`	Owns viewport + transform + `new_limits_event`. No interaction baked in.	Custom event wiring.
`AxesPanZoom`	Pure input controller — listens to `ViewportEvents`, mutates an `AxesDisplay`.	Pair with `AxesDisplay` for full control.

This is the same separation described in philosophy_examples.md §4.4. `vispy2` is where it lives.

Boundaries. No rendering, no backend dependency. Builds on `gsp` plus `gsp_extra`.

---

4. Extension Point — Adding a New Backend

The pattern is reproducible: every backend follows the same shape. To add `gsp_<backend>`:

1. **Mirror the layout** of gsp_matplotlib/:

```
src/gsp_<backend>/
├── __init__.py
├── renderer/
│   ├── __init__.py
│   └── <backend>_renderer.py        # subclass RendererBase
├── animator/
│   └── animator_<backend>.py        # subclass AnimatorBase
├── viewport_events/
│   └── viewport_events_<backend>.py # subclass ViewportEventsBase
└── renderer_registration.py
```

2. **Implement the three contracts** from src/gsp/types/:
   - `RendererBase` — concrete `render(viewports, visuals, model_matrices, cameras, **opts)`.
   - `AnimatorBase` — frame loop driving the renderer.
   - `ViewportEventsBase` — backend-specific input plumbing.
3. **Provide a `register_renderer_<backend>()` function** that calls `RendererRegistry.register_renderer(...)` — copy src/gsp_matplotlib/renderer_registration.py verbatim and rename.
4. **Wire into the example helper** at examples/common/example_helper.py:42 so `GSP_RENDERER=<backend>` becomes a valid value.

That is the full surface area. Existing examples will run under the new backend without modification.

---

5. Cross-Package Workflows

The architecture exists to make these three flows trivially expressible. Each shows multiple packages cooperating.

5.1 Local interactive plot

```
from gsp.core import Canvas, Viewport, Camera
from gsp.visuals import Points
from gsp_extra.bufferx import Bufferx
from gsp.utils.renderer_registery import RendererRegistry
from vispy2.axes import AxesPanZoom, AxesDisplay

import gsp_datoviz                                    # registers "datoviz"

canvas    = Canvas(width=512, height=512, dpi=127.5)
viewport  = Viewport(0, 0, 512, 512)
positions = Bufferx.from_numpy(np.random.rand(1000, 3), BufferType.vec3)
```

```
points    = Points(positions, ...)

renderer  = RendererRegistry.create_renderer("datoviz", canvas)
events    = RendererRegistry.create_viewport_events(renderer, viewport)
display   = AxesDisplay(viewport, ...)
panzoom   = AxesPanZoom(events, base_scale=1.1, axes_display=display)

renderer.render([viewport], [points], [Bufferx.mat4_identity()], [Camera(...)])
renderer.show()
```

Package map: `gsp` (Canvas/Viewport/Camera/Points), `gsp_extra` (Bufferx), `gsp_datoviz` (renderer + events), `vispy2` (AxesPanZoom/AxesDisplay).

5.2 Headless image export

Same construction code; **swap one import** for `gsp_matplotlib` and one string for `"matplotlib"`. The PNG bytes come back from `renderer.render(...)`. This is the demonstration that the env-var swap is real, not aspirational.

5.3 Remote render

```
[ user code ]                [ Flask server ]              [ rendering host ]
NetworkRenderer  ——HTTP——>  network_server.py  ———————>   MatplotlibRenderer
   uses                        uses                            OR
gsp_pydantic.PydanticSerializer  gsp_pydantic.PydanticParser   DatovizRenderer
                                                              (chosen per request)
```

Package map: client side uses `gsp_network` (renderer + serializer call) and `gsp_pydantic`. Server side uses `gsp_network.tools.network_server`, which itself imports `gsp_matplotlib` and `gsp_datoviz` at the top of network_server.py:19-20 and dispatches per `payload["renderer_name"]` at network_server.py:75-79.

The user's local code is identical to single-machine code — only the renderer constructor changes.

---

6. Stability Matrix

Package	Class	Guidance
`gsp`	**Stable contract**	Long-lived code should depend on this. Changes to abstract bases are versioned; changes to concrete `core`/`visuals` happen carefully.
`gsp_matplotlib`	**Stable backend**	The default backend; safe to depend on directly when you need backend-specific features (vector formats).
`gsp_datoviz`	**Stable backend**	Safe to depend on directly when you need GPU/interactive features.
`gsp_network`	**Stable backend**	Safe to depend on directly when you need remote rendering.

`gsp_extra`	**Convenience layer**	Free to depend on; helpers expected to grow but not to remove existing names without notice.
`gsp_pydantic`	**Convenience layer**	Schema may evolve as new visuals are added — pin a version if you persist serialized scenes long-term.
`vispy2`	**Convenience layer**	Higher-level API; expect more churn here than in `gsp`. Use the parallel-list `RendererBase.render(...)` API directly if you need maximum stability.

General rule. Code that needs to outlive a refactor should depend on `gsp` and use the registry to get a backend by name. Code that wants ergonomics over stability should reach into `gsp_extra` and `vispy2`.

---

7. Verification: how to read this document against the code

Every claim above is grounded in a specific file. To verify:

1. **The dependency table in §2.4.** For each `<pkg>` in { `gsp`, `gsp_matplotlib`, `gsp_datoviz`, `gsp_network`, `gsp_extra`, `gsp_pydantic`, `vispy2` }:

   ```
   grep -rn "^from gsp\|^import gsp" src/<pkg>/ | grep -oE "gsp[a-z_]*" | sort -u
   ```

   Each package's set of imports should match its row in the table.

2. **The contract implementations in §3.** For each backend:

   ```
   grep -rn "class .*\(RendererBase\)\|class .*\(AnimatorBase\)\|class .*\(ViewportEventsBase\)" src/gsp_<backend>/
   ```

   Three matches per backend, mirroring the table in each `Implements` block.

3. **The registration list in §2.2.**

   ```
   grep -rn "RendererRegistry.register_renderer" src/
   ```

   Exactly three matches: matplotlib, datoviz, network. If a fourth appears, §2.2 needs a new row.

4. **The extension recipe in §4.** Every step points at an existing file; copying src/gsp_matplotlib/ is the literal blueprint.

When the code drifts from this document — change the document. The package layout is the source of truth; this file just names what it already is.